Clustering and Outlier Detection Report

Name: Galip Alperen Aydınlık

1.Introduction

1.1 What is Clustering

Clustering is a Machine Learning technique that groups similar data points together.

Clustering helps discover hidden patterns and natural groupings in datasets by putting the similar data points in the same cluster.

It has a lot of methods to implement which we explain later on the report.

It uses formulas like Euclidean Distance to find distance between data points.

1.2 What is Outlier Detection

Outliers are data points that differs to much from other data points in the dataset.

For example a list hast 6 values: {1,2,3,4,5,60}

Here, 60 is an outlier value because it differs from other points.

Outlier Detection allows us to find these outlier values. There are many methods to implement Outlier Detection which will be explained later on the report.

---

2. Types of Clustering

2.1 Hard Clustering

Hard Clustering assigns each point exactly one cluster.

There are no overlap between clusters.

K-Means Clustering and Agglomerative Clustering are examples of this type of clustering.

2.2 Soft Clustering

Soft Clustering allows data points to belong multiple clusters. Unlike the Hard Clustering, it gives degree of membership to each cluster.

For example a data point can have multiple degrees like this (Next page)

```
Fuzzy Membership Matrix: (first 5 data points)
[[0.0753945  0.103008   0.03041289 0.09668828 0.07004986]
 [0.16579475 0.86917327 0.95209853 0.36321813 0.85265308]
 [0.75881075 0.02781873 0.01748858 0.54009359 0.07729706]]
```

Here, there are 5 Data Points and 3 clusters. Each column shows the degree of cluster on the given point.

In data point 1, third cluster is the most dominant one.

In data point 2, second cluster is the most dominant one.

Methods of Soft Clustering are Gaussian Mixture Model and Fuzzy-C Means.

---

3. Clustering Methods

3.1 K Means Clustering

K Means Clustering is a Hard Clustering algorithm that divides the dataset into k different clusters.

It is Commonly used in customer segmentation, image compression and pattern discovery.

Here are the steps of the K Means Clustering :

1. First, we randomly select k amount of centroids from data points.

2. Then each data point is assigned to nearest centroid, which forms clusters.

3. Centroids are updated by calculating the mean of the data points.

4. Process repeats until centroids dont change or we have reached the maximum amount of iteration.

On the next page, I will show you an one normal example and one real life example about the K Means Clustering method.

3.1.1 K Means Clustering Example

k_means_example_code.py

```python
import numpy as np # for distance calculation
from sklearn.datasets import make_blobs # to create a dataset
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

X,y = make_blobs(n_samples=500, n_features=2, centers=3, random_state=23)
# creates dataset with 500 entries, 2D environment, and 3 clusters

fig = plt.figure(0)
plt.grid(True) # adds an grid to plot
plt.scatter(X[:,0],X[:,1]) # scatter plot featuring x and y axis
plt.show()

scaler = StandardScaler()
X = scaler.fit_transform(X)
# scaling takes the mean and std deviation of data and converts into standard scale value.

clusters = {}
k = 3 # number of centroids
np.random.seed(23) # defined seed number means that we will always choose the same centroids

for i in range(k):
    center = 2*(2*np.random.random((X.shape[1])) - 1) # centroids are between positions -2 and 2
    points = []
    cluster = {'center': center, 'points': []}
    clusters[i] = cluster
# for loop used to create k amount of random centroids

print("\n")
print(clusters) # prints the location of each centroid (cluster)
```

k_means_example_code.py

```python

plt.scatter(X[:,0],X[:,1])
plt.grid(True)

for i in clusters:
    center = clusters[i]['center']
    plt.scatter(center[0], center[1], marker='*', c='red')
plt.show()
# Here, we plot the data points and centroids
# Centroids are in '*' shape and red color

def distance(p1:{__sub__}, p2):  2+ usages
    return np.sqrt(np.sum((p1-p2)**2))
# finding the distance between data points using euclidean formula

# these function created to assign clusters
def assign_clusters(X:{shape, __getitem__}, clusters:{__getitem__}):  1 usage
    for idx in range(X.shape[0]): # checks every row
        dist = [] # distance between current point and each centroid available
        curr_X  = X[idx] # current data point

        for i in range(k): # checks every cluster
            dis = distance(curr_X, clusters[i]['center']) # distance between current data point and center[i]
            dist.append(dis) # then, update the distance to list named dist[]

        curr_cluster = np.argmin(dist) # finds the index of smallest dist[i]
        clusters[curr_cluster]['points'].append(curr_X)
        # data point will be assigned to closest cluster

    return clusters
```

```
61
62      # updates the location of centroids by taking mean of the points in its surroundings
63      def update_clusters(X, clusters :{__getitem__} ):  1 usage
64          for i in range(k): # go through each centroid
65              points = np.array(clusters[i]['points']) # turns the cluster points to NumPy array so mean can found
66              if points.shape[0] > 0: # checks if cluster has point or not
67                  new_center = points.mean(axis=0)
68                  clusters[i]['center'] = new_center # new centroid defined
69
70                  clusters[i]['points'] = [] # clear point list
71
72          return clusters
73
74      # this function used to predict the centroid for each data point
75      def pred_cluster(X :{shape, __getitem__} , clusters :{__getitem__} ):  1 usage
76          pred = []
77          for i in range(X.shape[0]): # check every row
78              dist = [] # list for distance between the current data point and centroid
79              for j in range(k):
80                  dist.append(distance(X[i], clusters[j]['center']))
81              pred.append(np.argmin(dist)) # appends the minimum distance between centroid and data point to prediction list
82
83          return pred
84
85      clusters = assign_clusters(X, clusters)
86      clusters = update_clusters(X, clusters)
87      pred = pred_cluster(X, clusters)
88
89      plt.scatter(X[:,0],X[:,1], c=pred)
90      for i in clusters:
91          center = clusters[i]['center']
92          plt.scatter(center[0],center[1],marker= '^', c='red')
93
94      plt.show()
95
```

In this code, scikit-learn, matplot, and numpy libraries have been implemented.

Matplot library used to create plot graph, scikit-learn is used for creating a sample dataset and standart scaling for the dataset.

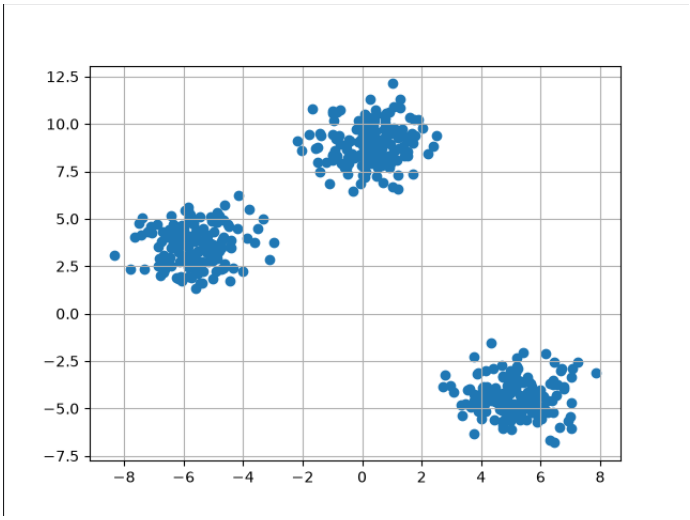
```
X,y = make_blobs(n_samples=500, n_features=2, centers=3, random_state=23)
# creates dataset with 500 entries, 2D environment, and 3 clusters

fig = plt.figure(0)
plt.grid(True) # adds an grid to plot
plt.scatter(X[:,0],X[:,1]) # scatter plot featuring x and y axis
plt.show()
```

Here, we used make_blops function to create an example dataset that has 3 centers with 500 data points.

Dataset is showed on the next page.

```
fig = plt.figure(0)
plt.grid(True) # adds an grid to plot
plt.scatter(X[:,0],X[:,1]) # scatter plot featuring x and y axis
plt.show()
```

By using plt function, we created an 2D plot with grid. Scatter function used to create x and y-axis.



As you can clearly see, dataset has 3 different groups which we assign centroids to each part later on the code.

```
scaler = StandardScaler()
X = scaler.fit_transform(X)
# scaling takes the mean and std deviation of data and converts into standard scale value.

clusters = {}
k = 3 # number of centroids
np.random.seed(23) # defined seed number means that we will always choose the same centroids
```

StandartScaler() function used before any clustering operation because if the features have different value ranges, one feature can effect distance calculation more than the others.

So mean and standard deviation of the data is converted to standard scale value.

After the scaling, we create an empty list called clusters to store information about clusters.

k=3 means number of centroids.

Np.random.seed(23) means seed no:23 choosen for the dataset and when we run the code more than once the output wont change.

You can think this seed number as an Minecraft seed, in Minecraft, each world has different seed and if you enter the same seed number, you will enter same world always.

```
for i in range(k):
    center = 2*(2*np.random.random((X.shape[1])) - 1) # centroids are between positions -2 and 2
    points = []
    cluster = {'center': center, 'points': []}
    clusters[i] = cluster
# for loop used to create k amount of random centroids

print("\n")
print(clusters) # prints the location of each centroid (cluster)

plt.scatter(X[:,0],X[:,1])
plt.grid(True)
```

Here, we used for loop k times to create 3 centroids.

center = 2*(2*np.random.random((X.shape[1])) - 1), this function makes centers position between -2 and 2

np.random.random((X.shape[1])) → 0 and 1

2*np.random.random((X.shape[1]) →0 and 2

(2*np.random.random((X.shape[1])) - 1) → -1 and 1

2*(2*np.random.random((X.shape[1])) - 1) →-2 and 2

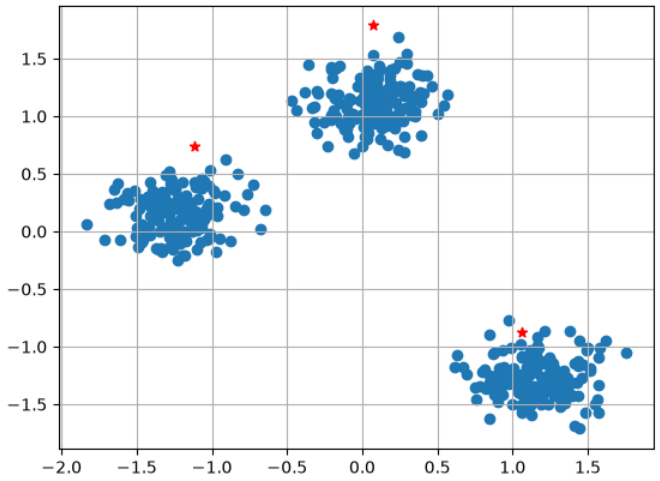
Empty points list created since we want to store data point information.

In clusters[] list, we keep the information of points and centroids.

```
for i in clusters:
    center = clusters[i]['center']
    plt.scatter(center[0], center[1], marker='*', c='red')
plt.show()
# Here, we plot the data points and centroids
# Centroids are in '*' shape and red color

def distance(p1 :{__sub__} , p2):  2+ usages
    return np.sqrt(np.sum((p1-p2)**2))
# finding the distance between data points using euclidean formula
```

In this for loop, centroids and points are plotted, and centroids showed with red color and '*' sign. (Graph at next page.)

Euclidean Distance formula in the distance function used to find the distance between data points.



As you can see from the graph, centroids are not in ther final location yet.

```python
# these function created to assign clusters
def assign_clusters(X :{shape, __getitem__} , clusters :{__getitem__} ): 1 usage
    for idx in range(X.shape[0]): # checks every row
        dist = [] # distance between current point and each centroid available
        curr_X  = X[idx] # current data point

        for i in range(k): # checks every cluster
            dis = distance(curr_X, clusters[i]['center']) # distance between current data point and center[i]
            dist.append(dis) # then, update the distance to list named dist[]

        curr_cluster = np.argmin(dist) # finds the index of smallest dist[i]
        clusters[curr_cluster]['points'].append(curr_X)
        # data point will be assigned to closest cluster

    return clusters
```

This function is created to assign clusters.

First we created an for loop that checks every row.

In the loop, an empty list called dist is created, it will store the distance between the current point and each centroid available.

Curr_X is the current data point.

Inside this loop, we will create another nested loop that checks every centroid.

Variable named dis created which is distance between current point and center.

Then, we will append this variable to dist list.

After that by using argmin(), we will find the index of the smallest dist value and append it to the clusters list.

This means we have assigned data point to the closest centroid.

```python
# updates the location of centroids by taking mean of the points in its surroundings
def update_clusters(X, clusters :{__getitem__} ):  1 usage
    for i in range(k): # go through each centroid
        points = np.array(clusters[i]['points']) # turns the cluster points to NumPy array so mean can found
        if points.shape[0] > 0: # checks if cluster has point or not
            new_center = points.mean(axis=0)
            clusters[i]['center'] = new_center # new centroid defined

            clusters[i]['points'] = [] # clear point list

    return clusters
```

In this function, for loop created to run through each centroid.

Inside the loop, points variable defined and np from numPy library used to store the data points inside an array.

İf condition is used to make sure update clusters that has at least one point.

By calculating the mean value of the points, we updated the centers then stored the new centers information inside clusters list.

At the end of the code, we cleared the point list.

```python
# this function used to predict the centroid for each data point
def pred_cluster(X :{shape, __getitem__} , clusters :{__getitem__} ):  1 usage
    pred = []
    for i in range(X.shape[0]): # check every row
        dist = [] # list for distance between the current data point and centroid
        for j in range(k):
            dist.append(distance(X[i], clusters[j]['center']))
        pred.append(np.argmin(dist)) # appends the minimum distance between centroid and data point to prediction list

    return pred
```

Here, prediction function used to predict the cluster label for each data point.

Since we already updated the centers, prediction is gonna be more accurate.

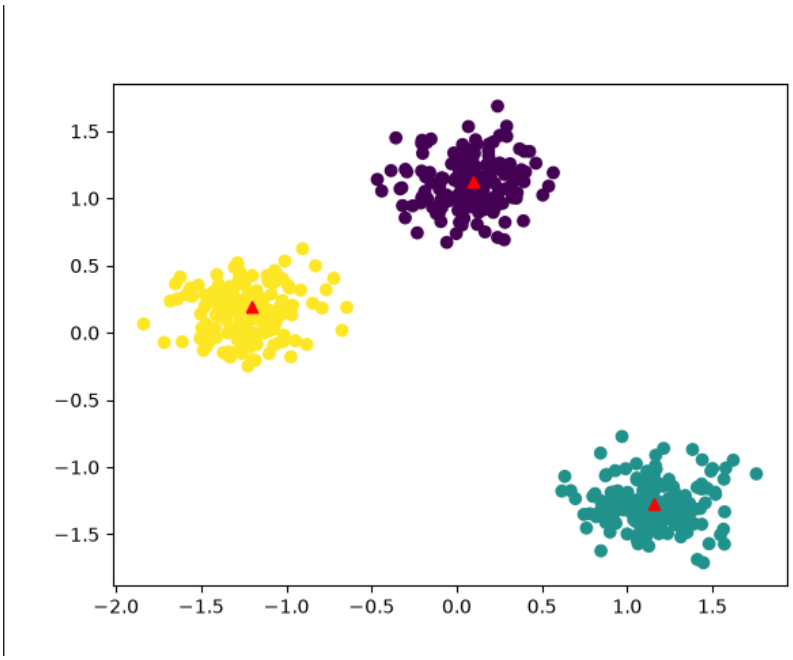
```python
clusters = assign_clusters(X, clusters)
clusters = update_clusters(X, clusters)
pred = pred_cluster(X, clusters)

plt.scatter(X[:,0],X[:,1], c=pred)
for i in clusters:
    center = clusters[i]['center']
    plt.scatter(center[0],center[1],marker= '^', c='red')

plt.show()
```

At lastly, we will implement all these functions that we created inside clusters list.

Then, we will create an plot that shows data points and center.

Center will be colored red and '^' sign is used.

Here is how the graph looks in the end:



3.1.2 K Means Example with Real Dataset

I have also implemented another K Means example but this time I have used an real dataset called 'Mall_Customers.csv' from Kaggle.

Since I used an real dataset, pandas library is added to the program.

Here is the libraries and main code is at the next page.

customer_example.py × Mall_Customers.csv

```
1  import numpy as np
2  from matplotlib import pyplot as plt
3  from sklearn.preprocessing import StandardScaler
4  import pandas as pd
```

Small part from the Dataset:

	CustomerID	Gender	Age	Annual Income (k…	Spending Score (…
1	1	Male	19	15	39
2	2	Male	21	15	81
3	3	Female	20	16	6
4	4	Female	23	16	77
5	5	Female	31	17	40

Main Code:

```
df = pd.read_csv('Mall_Customers.csv')
print(df.head())
print("\n")

print("Dataset Information")
print(df.info())
print("\n")

X_df = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# scale before KMeans
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_df)

k = 5 # number of centroids
clusters = {}
np.random.seed(23)

for i in range(k):
    center = 4 * (np.random.random(X_scaled.shape[1])) - 2 # between -2 and 2
    points = []
    cluster = {'center': center, 'points': []}
    clusters[i] = cluster

centers_array = np.array([clusters[i]['center'] for i in range(k)])
# array to keep all centroids

# First graph without centroids
fig = plt.figure()
plt.grid(True)
plt.scatter(X_scaled[:,0], X_scaled[:,1])
plt.show()

# Graphic with centroids
plt.scatter(X_scaled[:,0], X_scaled[:,1])
plt.grid(True)
plt.scatter(centers_array[:,0], centers_array[:,1], marker='x', color='red')
plt.show()
```

```python
def distance(p1 :{__sub__}, p2):  2+ usages
    return np.sqrt(np.sum((p1-p2)**2))

def assign_clusters(X :{shape, __getitem__}, clusters :{__getitem__}):  1 usage
    for idx in range(X.shape[0]): # go through every row
        dist = [] # dist between data point and cluster
        curr_X = X[idx] # current index

        for i in range(k): # go through every cluster
            dis = distance(curr_X, clusters[i]['center']) # calculate distance between cluster and current point
            dist.append(dis) # update the dist list

        curr_cluster = np.argmin(dist) # choose the minimum distance
        clusters[curr_cluster]['points'].append(curr_X) # data point assigned to closest cluster

    return clusters

def update_clusters(X, clusters :{__getitem__}):  1 usage
    for i in range(k):
        points = np.array(clusters[i]['points']) # puts cluster points into array using NumPy
        if points.shape[0] > 0:
            new_center = points.mean(axis=0)
            clusters[i]['center'] = new_center

        clusters[i]['points'] = []

    return clusters

def predict_clusters(X :{shape, __getitem__}, clusters :{__getitem__}):  1 usage
    pred = []
    for idx in range(X.shape[0]):
        dist = []
        for j in range(k):
            dist.append(distance(X[idx], clusters[j]['center']))

        pred.append(np.argmin(dist))

    return pred
```

```python
for i in range(20):
    clusters = assign_clusters(X_scaled, clusters)
    clusters = update_clusters(X_scaled, clusters)

pred = predict_clusters(X_scaled, clusters)

plt.scatter(X_scaled[:,0], X_scaled[:,1])
plt.grid(False)

for i in clusters:
    center = clusters[i]['center']
    plt.scatter(center[0],center[1],marker= '*', c='red')

plt.show()
```

```
df = pd.read_csv('Mall_Customers.csv')
print(df.head())
print("\n")

print("Dataset Information")
print(df.info())
print("\n")

X_df = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

Here, I read the file using read_csv() function and printed its information.

I decided not to take age and other unnecessary rows because it might affect the program wrong. Center locations might defined false because of it.

```
# scale before KMeans
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_df)

k = 5 # number of centroids
clusters = {}
np.random.seed(23)

for i in range(k):
    center = 4 * (np.random.random(X_scaled.shape[1])) - 2 # between -2 and 2
    points = []
    cluster = {'center': center, 'points': []}
    clusters[i] = cluster

centers_array = np.array([clusters[i]['center'] for i in range(k)])
# array to keep all centroids
```

Overall, this part is same with first program so no further explanation needed but only difference is, since we have 5 centroids, I have decided to put them on an array using numPy.

Next part of the code is mostly same too which it's showed in the next page.

```python
# First graph without centroids
fig = plt.figure()
plt.grid(True)
plt.scatter(X_scaled[:,0], X_scaled[:,1])
plt.show()

# Graphic with centroids
plt.scatter(X_scaled[:,0], X_scaled[:,1])
plt.grid(True)
plt.scatter(centers_array[:,0], centers_array[:,1], marker='x', color='red')
plt.show()

def distance(p1 : {__sub__} , p2):  2+ usages
    return np.sqrt(np.sum((p1-p2)**2))

def assign_clusters(X : {shape, __getitem__} , clusters : {__getitem__} ):  1 usage
    for idx in range(X.shape[0]): # go through every row
        dist = [] # dist between data point and cluster
        curr_X = X[idx] # current index

        for i in range(k): # go through every cluster
            dis = distance(curr_X, clusters[i]['center']) # calculate distance between cluster and current point
            dist.append(dis) # update the dist list

        curr_cluster = np.argmin(dist) # choose the minimum distance
        clusters[curr_cluster]['points'].append(curr_X) # data point assigned to closest cluster

    return clusters
```

```python

    def update_clusters(X, clusters : {__getitem__} ):  1 usage
        for i in range(k):
            points = np.array(clusters[i]['points']) # puts cluster points into array using NumPy
            if points.shape[0] > 0:
                new_center = points.mean(axis=0)
                clusters[i]['center'] = new_center

            clusters[i]['points'] = []

        return clusters

    def predict_clusters(X : {shape, __getitem__} , clusters : {__getitem__} ):  1 usage
        pred = []
        for idx in range(X.shape[0]):
            dist = []
            for j in range(k):
                dist.append(distance(X[idx], clusters[j]['center']))

            pred.append(np.argmin(dist))

        return pred

    for i in range(20):
        clusters = assign_clusters(X_scaled, clusters)
        clusters = update_clusters(X_scaled, clusters)

    pred = predict_clusters(X_scaled, clusters)

    plt.scatter(X_scaled[:,0], X_scaled[:,1])
    plt.grid(False)

    for i in clusters:
        center = clusters[i]['center']
        plt.scatter(center[0],center[1],marker= '*', c='red')

    plt.show()

```

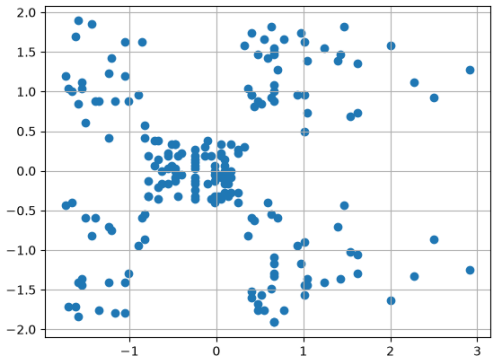
Only difference here is, I have implemented for loop at last part to update centers more than once to make it stable.

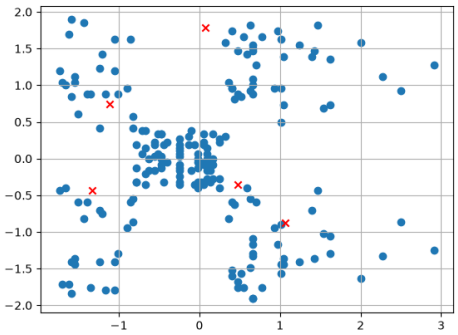
Here is 3 outputs of the plot,

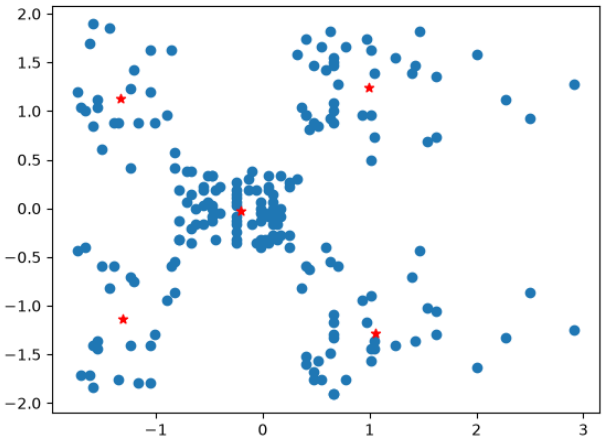
1- First Dataset

2- Dataset with randomly assigned centroid

3- Dataset with stable centroids







3.2 DBSCAN Clustering

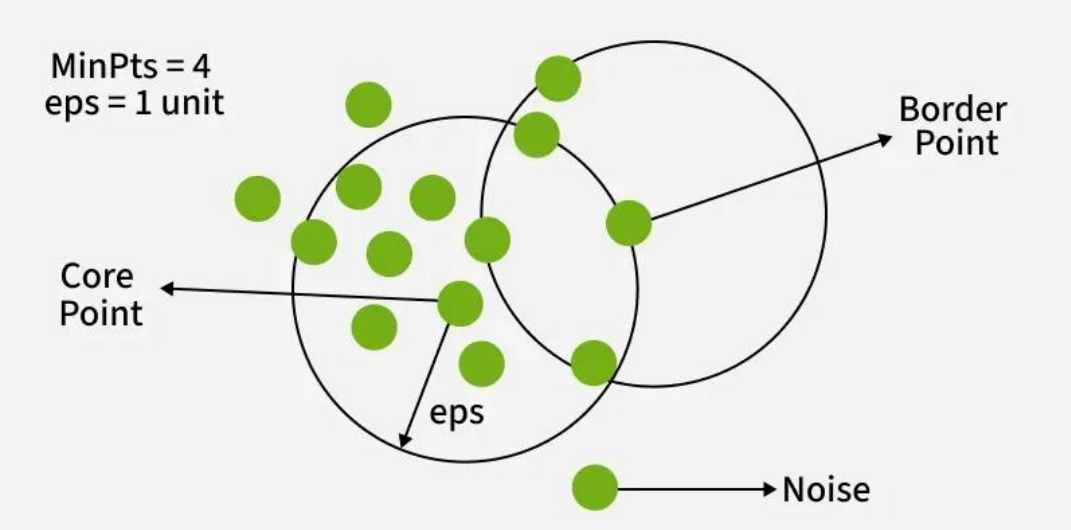
DBSCAN is a density-based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space.

Just like K Means Clustering its an Hard Clustering Method.

In DBSCAN, we implement core points.

Core points are point that have sufficient amount of neighbours around it.

Core points have an radius called eps (epsilon), and another points can be inside of it, on the border, or outside.



MinPts = 4 means that point must have at least 4 points to become core point.

If a point is not inside of an core point's radius, it is called noise and dont taken seriously because it can effect data accuracy wrongly.

For each core point that is not already assigned to a cluster create a new cluster. Recursively find all density-connected points i.e points within the eps radius of the core point and add them to the cluster.

DBSCAN Clustering Examples will be given on the next page.

First normal example with sample dataset, then real life example that uses Airbnb listings as dataset.

3.2.1 DBSCAN Clustering Example

There are many libraries used in the DBSCAN just like K-means. From sklearn, we have implement DBSCAN function that helps us create an DBSCAN algorithm.

DBSCAN_EXAMPLE.py

```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn import metrics
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn import datasets
from sklearn.metrics import adjusted_rand_score
```

```python
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.50, random_state=0)
# cluster_std controls how spread out each cluster is

db = DBSCAN(eps=0.3, min_samples=10).fit(X) # eps is the radius of each core point
# to define core point, point must have at least 10 neighbors
```

Dataset with 300 samples and 4 centers is created.

Additionaly cluster_std is given which controls how spread out each cluster is.

After that, we created variable named db and assigned it to DBSCAN function.

In DBSCAN, eps is 0.3 which is core point and to define a core point, it must have an at least 10 neighbors.

```python
core_samples_mask = np.zeros_like(db.labels_, dtype=bool) # creates an array where all values are false
core_samples_mask[db.core_sample_indices_] = True # changes core points to True
labels = db.labels_ # labels are cluster numbers, if -1 then it's a noise
```

Here, we created variables called core_samples_mask and labels.

NumPy library used to create an array where all values are false.

By using variable core_sample_indices, we changed all the core values in array to true.

Labels are cluster numbers, if cluster number equals to -1, that means it's a noise (outlier value).

```
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
# gives number of clusters, if there is a -1 in the labels, subtract them

unique_labels = set(labels) # Here, we prepare labels and colors
colors = ['y','b','g','r']
print(colors)
```

The amount of clusters are found keeped in variable called n_clusters.

Different colors for each cluster defined and then printed.

```
for k, col in zip(unique_labels, colors):
    if k == -1: # if the point is noise paint it black
        col = 'k'

    class_member_mask = (labels == k) # True / False mask for current cluster
    xy = X[class_member_mask & core_samples_mask] # Select points which currently in cluster and core point
    plt.plot( *args: xy[:,0], xy[:,1], 'o', markerfacecolor=col, markeredgecolor='k', markersize=6)
    # xy[:,0], xy[:,1] means x and y axis
    # uses circle markers and edge colors are black

    xy = X[class_member_mask & ~core_samples_mask]
    plt.plot( *args: xy[:,0], xy[:,1], 'x', markerfacecolor=col, markeredgecolor='k', markersize=6)
    # points where current cluster is not core point, could be border points
```

In for loop, we give each cluster a different color, but if noise detected, it will be colored black.

xy is the points that currently in cluster and a core point.

Then, we plotted xy with shape ‘o’ and each cluster different color.

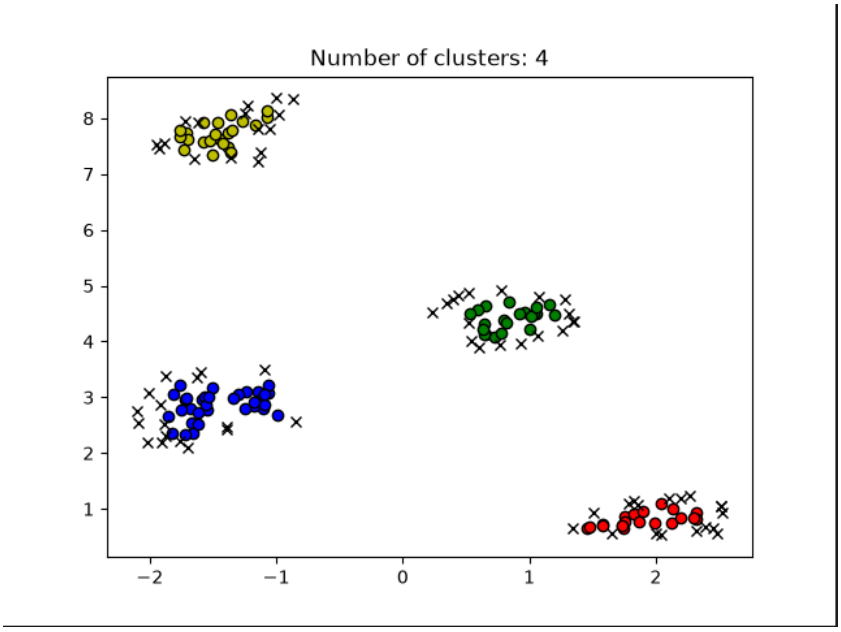
After that we defined xy as a points that are not core point which called noise.

Noise is marked with sign ‘x’ and it will be colored in black.

```
plt.title(f'Number of clusters: {n_clusters}')
plt.show()

sc = metrics.silhouette_score(X, labels)
print(f'Silhouette Coefficient: {sc}')
ari = metrics.adjusted_rand_score(y, labels)
print(f'Adjusted Rand Index: {ari}')
```

Lastly, we plotted the graph and also find Silhouette Coefficient. It checks whether points are well placed inside their clusters.

Here is plot output:



3.2.2 DBSCAN Clustering Using Prague Airbnb Locations

In this project, the DBSCAN clustering algorithm was applied to the Airbnb Prague dataset. The aim was to group Airbnb listings based on their geographic locations using latitude and longitude values.

Official dataset has been obtained from Airbnb website itself and it's the data from 2025, September.

By using libraries GeoPandas and Contextily, I have managed to put real Prague map into plot graph.

Additionaly, pandas library is implemented to read the Airbnb dataset.

Here is some part of the dataset:

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
1	id	listing_url	scrape_id	last_scraped	source	name	description	neighborhood_overview	picture_url	host_i
2	23163	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	previous scrape	Residence Karolina - KAROL12	Unique and elegant apartment…	<null>	https://a0.muscache.com/pict…	5282
3	23169	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	city scrape	Residence Masna - Masna302	Masna studio offers a lot of…	<null>	https://a0.muscache.com/pict…	5282
4	26755	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	city scrape	Central Prague Old Town Top …	Big and beautiful new attic …	This apartment offers a fant…	https://a0.muscache.com/pict…	113902
5	30762	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	city scrape	Residence Rybna - Rybna23	We offer a modern, comfortab…	<null>	https://a0.muscache.com/pict…	5282
6	42514	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	city scrape	"NEWLY FURNISHED" 1BR near P…	*NEWLY* Furnished 1-bedroom …	<null>	https://a0.muscache.com/pict…	185641
7	52148	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	city scrape	Colorful 2BDR Family Apartme…	-The apartment has a self-ch…	Nestled seamlessly between P…	https://a0.muscache.com/pict…	227945
8	55856	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	city scrape	Renovated Apartment behind N…	Cozy and well-equipped, this…	Nestled at the crossroads of…	https://a0.muscache.com/pict…	227945
9	75298	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	city scrape	Prague Studio Old Town	GOOD MORNING :) , I M Emanue…	<null>	https://a0.muscache.com/pict…	398991
10	79373	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	city scrape	Residence Masna - Masna201	Situated in the centre of Pr…	<null>	https://a0.muscache.com/pict…	5282
11	79381	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	previous scrape	Residence Masna - Masna504	Freshly renovated one bedroo…	<null>	https://a0.muscache.com/pict…	5282
12	79387	https://www.airbnb.com/rooms…	20250923203415	2025-09-24	previous scrape	Residence Masna - Masna203	Masna apartment rental is lo…	<null>	https://a0.muscache.com/pict…	5282

It is quite an big dataset so we need the prepare it before applying any clustering algorithm.

Main Code:

dbscan_prague.py | dbscan_prague_airbnb_map.png | listings.csv

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
import geopandas as gpd
import contextily as cx

df = pd.read_csv('listings.csv') # Airbnb Prague Dataset

print("First 5 Rows of Dataset")
print(df.head())
print("\n")

print("Dataset Information")
print(df.info())
print("\n")

print("Missing Values")
print(df.isnull().sum())
print("\n")

df = df[["name", "neighbourhood_cleansed", "latitude", "longitude", "room_type", "price"]]

print("Dataset with Selected Columns")
print(df.head())
print("\n")

print("Missing Values After Selecting Columns")
print(df.isnull().sum())
print("\n")

df = df.dropna(subset=['latitude', 'longitude'])
# delete rows where latitude and longitude empty

df["price"] = (df["price"].astype(str).str.replace( pat: "$", repl: "", regex=False))
df["price"] = (df["price"].astype(str).str.replace( pat: ",", repl: "", regex=False))
# changing how price look 2,500 -> 2500 || 3,400$ --> 3400

df["price"] = pd.to_numeric(df["price"], errors='coerce')
# if price can not converted to number, give NaN (missing value)
```

```python
print("Dataset After Cleaning")
print(df.head())
print("\n")

print("Missing Values After Cleaning")
print(df.isnull().sum())
print("\n")

coordinates = df[["latitude", "longitude"]].to_numpy() # creates a numPy array from latitude, longitude rows
```

```
print("Coordinate Date:")
print(coordinates[:5])
print("\n")

print("Coordinate Shape:")
print(coordinates.shape)
print("\n")

# Convert coordinates to radians for haversine distance
# Haversine distance is distance between two locations by looking at their longitude and latitude
coordinates_radians = np.radians(coordinates)
# radians = degrees × π / 180, we convert the coordinates values to radians so Haversine Distance can be applied

print("Coordinates in  Radians")
print(coordinates_radians[:5])
print("\n")

earth_radius_km = 6371.01
eps_km = 0.5
min_samples = 14

dbscan = DBSCAN(eps=eps_km / earth_radius_km, min_samples=min_samples, metric = 'haversine')

labels = dbscan.fit_predict(coordinates_radians) # creates cluster label for every listing
df["cluster"] = labels # add a new row named cluster labels
# if label = -1, it means that it's a noise

print("Cluster Labels")
print(df["cluster"].value_counts())
print("\n")

num_of_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print("Number of Clusters: ", num_of_clusters)
print("\n")

gdf = gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df["longitude"], df["latitude"]), crs='EPSG:4326')
# We convert normal dataframe to GeoDataFrame
```

```
print("GeoDataFrame")
print(gdf.head())
print("\n")

gdf_map = gdf.to_crs(epsg=3857) # convert GeoDataFrame to real map background

fig, ax = plt.subplots(figsize=(10,8))

gdf_map[gdf_map["cluster"] != -1].plot( # details for clusters
    ax=ax,
    column="cluster",
    cmap="tab20", # give different color for each cluster
    markersize=6,
    marker="o",
    alpha=0.8, # transparency rate, 1-> non transparent, 0.3-> very transparent
    legend=True
)
```

```
gdf_map[gdf_map["cluster"] == -1].plot( # details for noises
    ax=ax,
    color='black',
    markersize=4,
    marker="x",
    alpha=0.8,
    label='noise'
)

cx.add_basemap( # real map background
    ax,
    source=cx.providers.OpenStreetMap.Mapnik,
    crs = gdf_map.crs,
    zoom = 12
)

cluster_price = df.groupby("cluster")["price"].mean()
print("Average Price Per Cluster")
print(cluster_price)
print("\n")

ax.set_title("DBSCAN Clustering Using Prague Airbnb Listings")

ax.legend()
ax.grid(True)
ax.set_axis_off()
plt.savefig( fname: "dbscan_prague_airbnb_map.png", dpi=300, bbox_inches="tight")
plt.show()
```

As you can see from the code, contextility and GeoPandas library is used frequently so we can plot graph using Prague map as I mentioned before.

I will explain the code at the next page.

dbscan_prague.py | dbscan_prague_airbnb_map.png | listings.csv

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
import geopandas as gpd
import contextily as cx

df = pd.read_csv('listings.csv') # Airbnb Prague Dataset

print("First 5 Rows of Dataset")
print(df.head())
print("\n")

print("Dataset Information")
print(df.info())
print("\n")

print("Missing Values")
print(df.isnull().sum())
print("\n")
```

Firstly, we implemented the necessary library and then by using read_csv(), data has been read.

Dataset Information, and missing values from the dataset is printed out.

```python
df = df[["name", "neighbourhood_cleansed", "latitude", "longitude", "room_type", "price"]]

print("Dataset with Selected Columns")
print(df.head())
print("\n")

print("Missing Values After Selecting Columns")
print(df.isnull().sum())
print("\n")

df = df.dropna(subset=['latitude', 'longitude'])
# delete rows where latitude and longitude empty

df["price"] = (df["price"].astype(str).str.replace( pat: "$", repl: "", regex=False))
df["price"] = (df["price"].astype(str).str.replace( pat: ",", repl: "", regex=False))
# changing how price look 2,500 -> 2500 || 3,400$ --> 3400

df["price"] = pd.to_numeric(df["price"], errors='coerce')
# if price can not converted to number, give NaN (missing value)

print("Dataset After Cleaning")
print(df.head())
print("\n")

print("Missing Values After Cleaning")
print(df.isnull().sum())
print("\n")
```

We choosed only the necessary columns for clustering. Latitude and longitude are one of them because they will help us find distance between each Airbnb point and location of that point in real Prague map.

Then we dropped the rows that doesn't include latitude and longitude.

After that, we changed how prices look on the dataset so we can work with them.

Ex: 2,500$ → 2500 (readable for the system)

To.numeric() from pandas library is used so we can give NaN (missing value) to values that can not be converted to number.

```
coordinates = df[["latitude", "longitude"]].to_numpy() # creates a numPy array from latitude, longitude rows

print("Coordinate Date:")
print(coordinates[:5])
print("\n")

print("Coordinate Shape:")
print(coordinates.shape)
print("\n")

# Convert coordinates to radians for haversine distance
# Haversine distance is distance between two locations by looking at their longitude and latitude
coordinates_radians = np.radians(coordinates)
# radians = degrees × π / 180, we convert the coordinates values to radians so Haversine Distance can be applied

print("Coordinates in  Radians")
print(coordinates_radians[:5])
print("\n")
```

numPy array is created to store the coordinates of each Airbnb room.

We printed out the coordinates and their shape.

From numPy library, radians is used to convert coordinates to radians so we can use haversine distance.

The Haversine distance is the shortest distance between two points on the surface of a sphere, calculated using their latitude and longitude.

Coordinates in radians is printed.

```
earth_radius_km = 6371.01
eps_km = 0.5
min_samples = 14

dbscan = DBSCAN(eps=eps_km / earth_radius_km, min_samples=min_samples, metric = 'haversine')
```

Since we use a real map, earth radius must be defined.

Eps is defined as 500m and point must have an at least 14 points to be considered as core point.

```
labels = dbscan.fit_predict(coordinates_radians) # creates cluster label for every listing
df["cluster"] = labels # add a new row named cluster labels
# if label = -1, it means that it's a noise

print("Cluster Labels")
print(df["cluster"].value_counts())
print("\n")

num_of_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print("Number of Clusters: ", num_of_clusters)
print("\n")

gdf = gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df["longitude"], df["latitude"]), crs='EPSG:4326')
# We convert normal dataframe to GeoDataFrame

print("GeoDataFrame")
print(gdf.head())
print("\n")
```

By using fit_predict(), we created a cluster label for every listing.

Then we created a new row called cluster in dataset to store cluster labels.

Num_of_clusters() function used to find total amount of clusters, same formula has been used in previous DBSCAN example.

If -1(noise) present, dont add it to clusters.

GeoDataFrame() function allowed us to convert our dataframe to GeoDataFrame so we can use real map.

X_axis is longtiude and y_axis is latitude.

```
gdf_map = gdf.to_crs(epsg=3857) # convert GeoDataFrame to real map background

fig, ax = plt.subplots(figsize=(10,8))

gdf_map[gdf_map["cluster"] != -1].plot( # details for clusters
    ax=ax,
    column="cluster",
    cmap="tab20", # give different color for each cluster
    markersize=6,
    marker="o",
    alpha=0.8, # transparency rate, 1-> non transparent, 0.3-> very transparent
    legend=True
)

gdf_map[gdf_map["cluster"] == -1].plot( # details for noises
    ax=ax,
    color='black',
    markersize=4,
    marker="x",
    alpha=0.8,
    label='noise'
)

cx.add_basemap( # real map background
    ax,
    source=cx.providers.OpenStreetMap.Mapnik,
    crs = gdf_map.crs,
    zoom = 12
)
```

To_crs() is used to convert GeoDataFrame to real background.

EPSG:3857 (WGS 84 / Pseudo-Mercator) is a global projection system universally used by web mapping services such as Google Maps, OpenStreetMap, and Bing Maps.

First, we create the plot details for the clusters, then noise details are created.

cx.add_basemap is used for real map background.

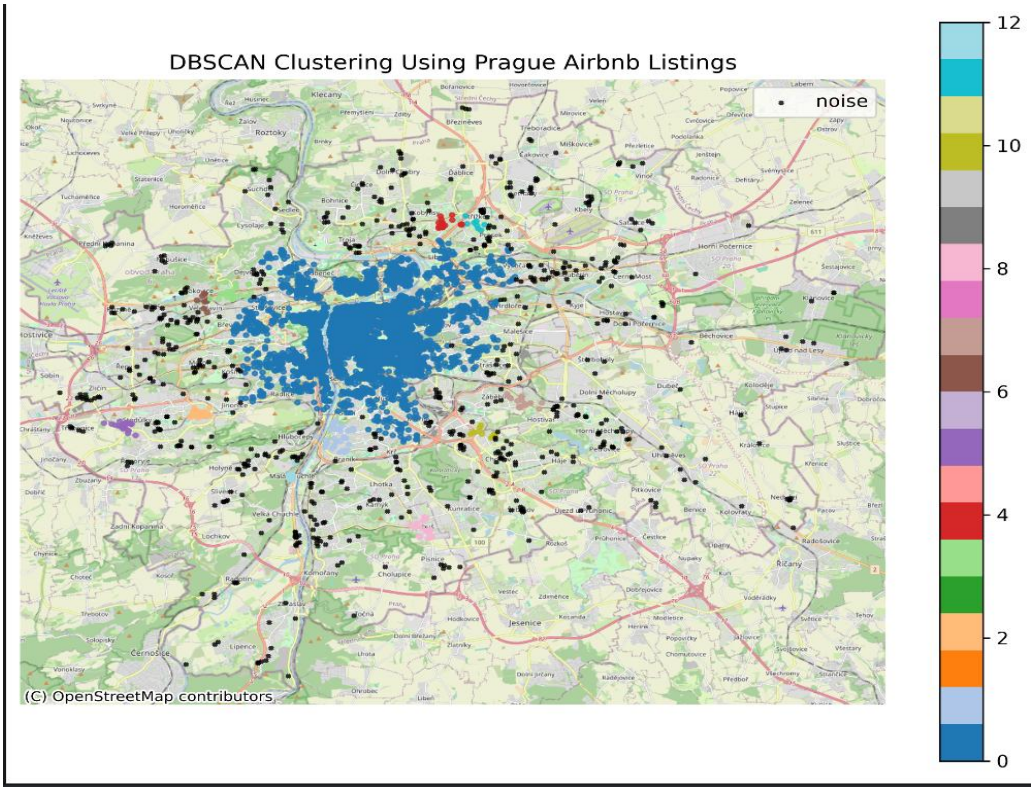
```
cluster_price = df.groupby("cluster")["price"].mean()
print("Average Price Per Cluster")
print(cluster_price)
print("\n")

ax.set_title("DBSCAN Clustering Using Prague Airbnb Listings")

ax.legend()
ax.grid(True)
ax.set_axis_off()
plt.savefig( fname: "dbscan_prague_airbnb_map.png", dpi=300, bbox_inches="tight")
plt.show()
```

Average Price per cluster is found by using mean() function.

After that, we started plotting the graph with legend and grid.

Lastly, image of the output is saved using savefig().

Here is the final output:



Blue points are the river side of the city which is the most popular place for Airbnb's.

Black points are outside of the city, places that doesnt attract many tourists.

3.3 Gaussian Mixture Model

Gaussian Mixture Model (GMM) is a probabilistic clustering technique that models data as a combination of multiple Gaussian distributions, allowing more flexible grouping of data points.

Gaussian Mixture Model is an Soft Clustering algorithm which means a point can belong to multiple clusters.

- Assigns each data point a probability of belonging to different clusters.
- Uses mean and covariance to define cluster shape.

Here is a probability function that gives probability of data point x belonging to cluster k.

$$P(z_n = k \mid x_n) = \frac{\pi_k \cdot \mathcal{N}(x_n|\mu_k,\Sigma_k)}{\sum_{j=1}^{K} \pi_j \cdot \mathcal{N}(x_n|\mu_j,\Sigma_j)}$$

In GMM, each cluster is a Gaussian defined by their mean and covierance.

Mean is center of cluster and covierance controls the shape of a cluster.

Visualizing GMM often involves scattering plots to show data.

gmm_example.py

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn.mixture import GaussianMixture
4  from sklearn.datasets import make_blobs
```

Here, sicikit-learn library is implemented because we want to create a sample data and use GMM model.

Code is at the next page:

```python
X, y = make_blobs(n_samples=500, centers=3, random_state=42, cluster_std=[1.0, 1.5, 0.8])
# 500 data points, 3 centroids, and spread of each cluster

gmm = GaussianMixture(n_components=3, covariance_type='full', random_state=42) # create 3 components
# covariance_type='full' means that cluster can have any shape
gmm.fit(X) # train the given data
labels = gmm.predict(X) # cluster labels assigned to each data
# GMM is a soft clustering algorithm, but predict will give the most high percentile cluster

plt.figure(figsize=(8,6))
plt.scatter(X[:,0], X[:,1], c=labels, cmap='viridis', s=50, edgecolors='k')
# s=50 meant size of the points

plt.scatter(
    gmm.means_[:, 0],
    gmm.means_[:, 1],
    c='red',
    marker='X',
    label='Centers'
)

# plotting the dataset
plt.title('Gaussian Mixture Model')
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.legend()
plt.show()
```

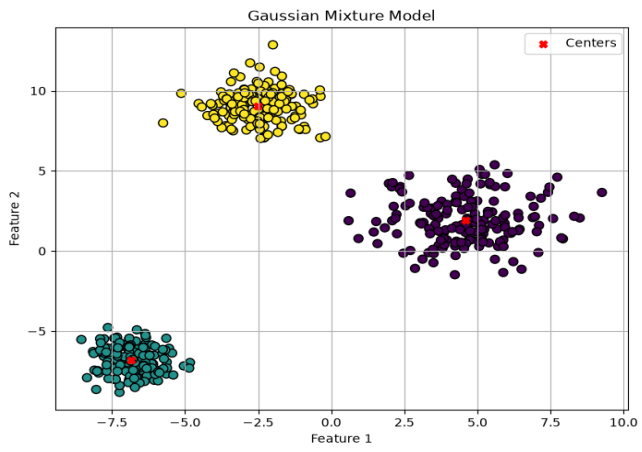
Gmm is defined to create an GMM example with 3 components.

Coveriance_type = ‘full’ means that clusters can be any shape.

Gmm.fit() trains the data and by using predict(), cluster labels assigned to each point.

After that we plotted the graph and find each center by using mean.

Here is the output:



3.3.1 GMM Using Wine Dataset from Scikit-Learn

In this example, GMM is implemented using Wine Dataset from scikit-learn library.

Additionaly, we used StandartScaler() and PCA functions from scikit-learn.

PCA is used to lower the amount of features which is 13 to only 2 because we are working on 2D Space.

Here is all the libraries used in this example:

wine_example.py

```python
from sklearn.datasets import load_wine # wine dataset from scikit-learn
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA # PCA used to lower the amount of features which is 13 to just 2 PCA features
```

And here is the full code:

```python
wine = load_wine()

x = wine.data
y = wine.target

print("Feature Names: ", wine.feature_names)
print("Target Names: ", wine.target_names)
print("Data Shape: ", x.shape)
print("Target Shape: ",y.shape)
print("\n")

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
# We need to scale data before any important operation

pca = PCA(n_components=2) # we still use 13 features but its converted to only 2 features since we plot dataset in 2D Shape
x_pca = pca.fit_transform(x_scaled)
print(x_pca.shape)
print("\n")

gmm = GaussianMixture(n_components=3, covariance_type='full', random_state=42)
gmm.fit(x_pca) # train the Wine Dataset
labels = gmm.predict(x_pca) # predict the cluster label
```

```python
plt.figure(figsize=(8,6))
plt.scatter(x_pca[:,0], x_pca[:,1], c=labels, cmap='viridis', s=50, edgecolors='k')

plt.scatter(
    gmm.means_[:, 0],
    gmm.means_[:, 1],
    c='red',
    marker='X',
    label='GMM_Means',
    s=100
)
```

```
plt.title("Gaussian Mixture Model With Wine Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True)
plt.show()
```

Since we want to add Wine Dataset, load_wine() function is defined in wine variable.

The variable x stores the feature values of the wine samples, while y stores the original class labels.

Then feature names, target names, and shape information is printed out.

After that, we scaled the data using StandartScaler() because wine dataset contains features with different value ranges.

```
pca = PCA(n_components=2) # we still use 13 features but its converted to only 2 features since we plot dataset in 2D Shape
x_pca = pca.fit_transform(x_scaled)
print(x_pca.shape)
print("\n")
```

Here, PCA function used to convert 13 features to 2 since we want to plot dataset in 2D shape.

Note: 2 features are combined versions of total 13 features.

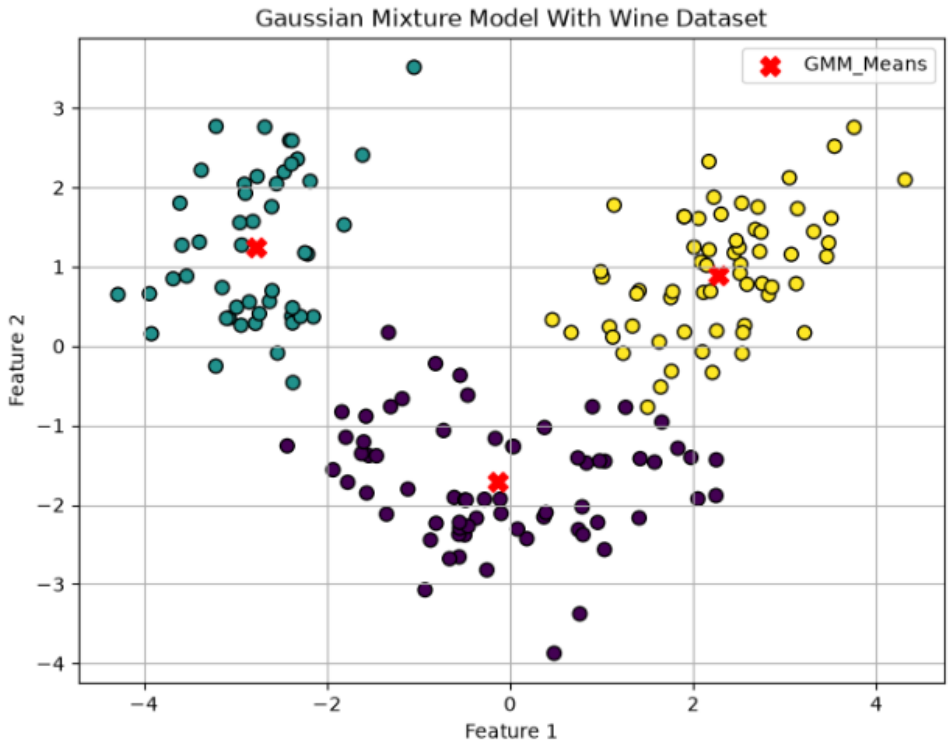
```
gmm = GaussianMixture(n_components=3, covariance_type='full', random_state=42)
gmm.fit(x_pca) # train the Wine Dataset
labels = gmm.predict(x_pca) # predict the cluster label
```

This part is same with previous Gaussian example. It uses 3 centers and every cluster shape is allowed.

Dataset is trained and cluster label for each point is predicted.

At last, we plotted the graph with centers that shows mean points and created each cluster.

Output is at the next page:



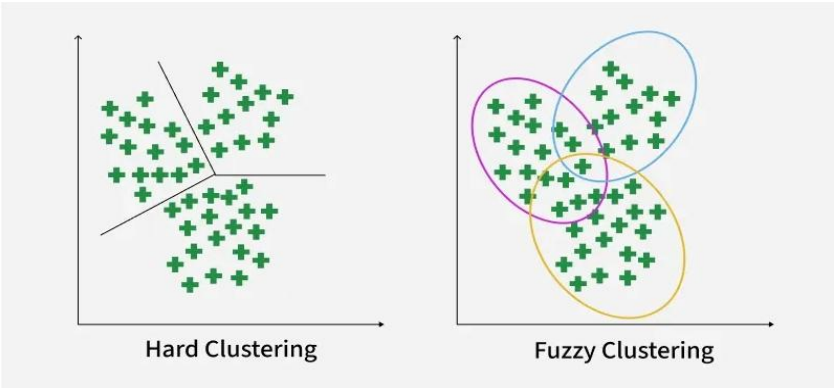
Wine dataset is more spread out than the dataset from the previous example.

3.4 Fuzzy-C Clustering Method

Fuzzy clustering allows each data point to belong to multiple clusters with different membership values. Instead of assigning a point to just one group, it captures how strongly a point relates to each cluster.

It is an Soft Clustering algorithm which means point can belong to multiple clusters.

It uses membership scores between 0 and 1.



There are 4 steps to implement Fuzzy-C Algorithm.

- First, we assign each data point a random membership score.
- Then, cluster centroids are calculated as weighted averages.
- Euclian Distance between each point and centroid is calculated.
- Lastly, by looking through each distance, membership scores are updated.

These steps continue until dataset becomes stable.

As a new library, scikit-fuzzy is implemented to show Fuzzy-C Clusterning.

Here is all the libraries used.:

```
from matplotlib import pyplot as plt
import numpy as np
import skfuzzy as fuzz
```

Main Code:

```
np.random.seed(0) # seed number given so result will be same each time code run
center = 0.5
spread = 0.1

data = center + spread * np.random.randn(2,100) # 100 data points in 2D space
data = np.clip(data, a_min: 0, a_max: 1) # data point are between 0 and 1

# Fuzzy-C Parameters
n_clusters = 3
m = 1.7 # higher this value, cluster memberships will be softer
error = 1e-5 # stopping condition, if the value change is smaller than this value, algorithm stops running.
maxiter = 2000 # maximum number of iterations (how many times code can run)

cntr, u, _, _, _, _, fpc = fuzz.cluster.cmeans(data, c=n_clusters, maxiter=maxiter, m=m,
                                               error=error, init=None)
# cntr is the final cluster centers
# u is a matrix indicates degree of belonging of point to each cluster

hard_clusters = np.argmax(u,axis=0) # we taken the highest value from each point's degree to define one hard cluster
                                    #for each data point
```

```
print("Cluster Centers: ")
print(cntr)
print("\n")

print("Fuzzy Membership Matrix: (first 5 data points)")
print(u[:,:5])
print("\n")

print("Hard Clusters of first 5 data points: ")
print(hard_clusters[:5])
print("\n")
```

```
fig, ax = plt.subplots(figsize=(8,6))

for i in range(n_clusters):
    ax.scatter(data[0], data[1], c=u[i], cmap='coolwarm', alpha=0.5, label=f'Fuzzy Cluster: {i+1}') #alpha = transparency rate
# c = u[i] means that points colored depend on how strongly they belong to each cluster
# For example if data point has the highest degree on cluster 2, it means that cluster 2 will be most colored

markers = ['o','s','^']
colors = ['b','g','orange'] # blue, green, orange for clusters 1,2,3

for i in range(n_clusters):
    cluster_points = data[:, hard_clusters == i]
    ax.scatter(cluster_points[0], cluster_points[1], c=colors[i], marker=markers[i],
               edgecolors='k', s=80, label=f'Hard Cluster: {i+1}')

ax.scatter(cntr[:,0], cntr[:,1], c='red', marker='x', s=200, label='Cluster Centers')

ax.set_title("Fuzzy C-Means")
ax.set_xlabel("Feature 1")
ax.set_ylabel("Feature 2")
ax.legend(loc='upper left') # legend will be in the upper left
plt.show()
```

First, we added random seed(0) and defined center, spread.

Center means that data point are created around 0.5, and spread means how much points are spread out from centers.

```
data = center + spread * np.random.randn(2,100) # 100 data points in 2D space
data = np.clip(data, a_min: 0, a_max: 1) # data point are between 0 and 1

# Fuzzy-C Parameters
n_clusters = 3
m = 1.7 # higher this value, cluster memberships will be softer
error = 1e-5 # stopping condition, if the value change is smaller than this value, algorithm stops running.
maxiter = 2000 # maximum number of iterations (how many times code can run)
```

After that we created a variable named data to create our dataset's parameters.

Dataset has 100 point in a 2D space and data points are between 0 and 1.

Fuzzy-C parameters are implemented.

There will be 3 number of clusters.

m determines how soft or hard membership scores will become. If m is smaller, this means that membership scores become softer.

So instead of saying that point 1 belongs to %100 to cluster A, we will say that point 1 belongs to multiple clusters with different membership scores.

Error rate is defined as a stopping condition. If the value change is smaller than 1e-5, algorithm will stop running.

Maxiter is the maximum number of iterations allowed.

```
cntr, u, _, _, _, _, fpc = fuzz.cluster.cmeans(data, c=n_clusters, maxiter=maxiter, m=m,
                                               error=error, init=None)
# cntr is the final cluster centers
# u is a matrix indicates degree of belonging of point to each cluster

hard_clusters = np.argmax(u,axis=0) # we taken the highest value from each point's degree to define one hard cluster
                                    #for each data point
```

Here, by using skfuzzy we have succesfully implemented the Fuzzy-C Algorithm.

Then, we found each points hard cluster (cluster with biggest membership score) by using argmax function from numPy library.

```
fig, ax = plt.subplots(figsize=(8,6))

for i in range(n_clusters):
    ax.scatter(data[0], data[1], c=u[i], cmap='coolwarm', alpha=0.5, label=f'Fuzzy Cluster: {i+1}') #alpha = transparency rate
# c = u[i] means that points colored depend on how strongly they belong to each cluster
# For example if data point has the highest degree on cluster 2, it means that cluster 2 will be most colored

markers = ['o','s','^']
colors = ['b','g','orange'] # blue, green, orange for clusters 1,2,3

for i in range(n_clusters):
    cluster_points = data[:, hard_clusters == i]
    ax.scatter(cluster_points[0], cluster_points[1], c=colors[i], marker=markers[i],
               edgecolors='k', s=80, label=f'Hard Cluster: {i+1}')

ax.scatter(cntr[:,0], cntr[:,1], c='red', marker='x', s=200, label='Cluster Centers')

ax.set_title("Fuzzy C-Means")
ax.set_xlabel("Feature 1")
ax.set_ylabel("Feature 2")
ax.legend(loc='upper left') # legend will be in the upper left
plt.show()
```

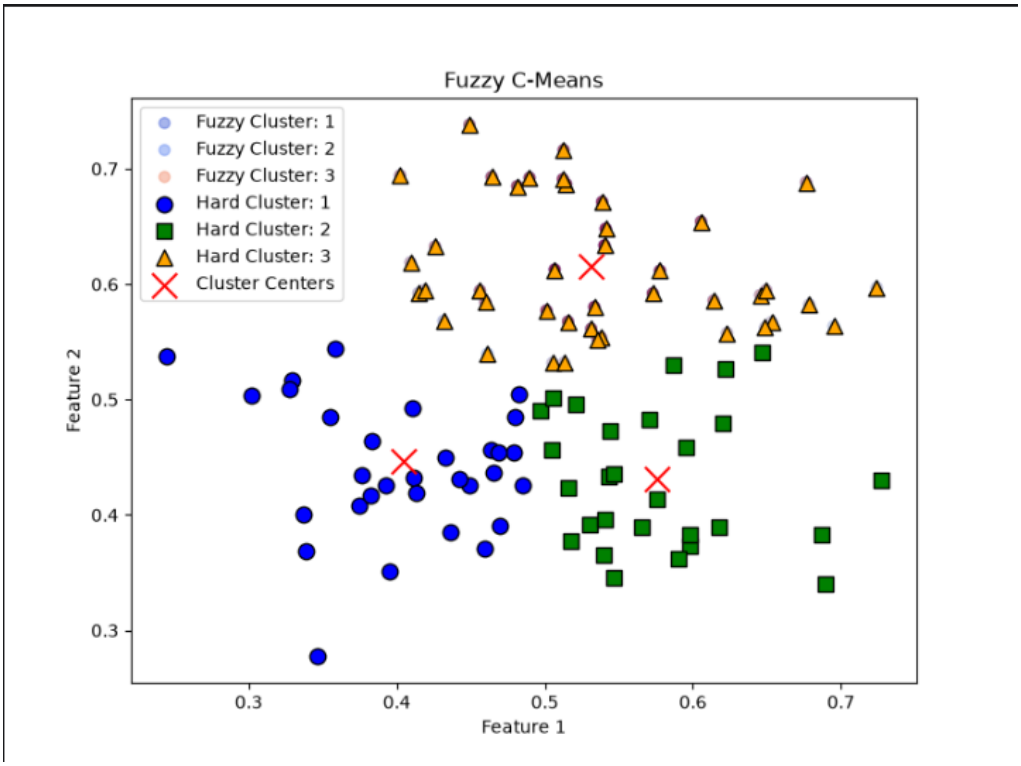
In the first for loop we scattered each data point with transparency rate (alpha) 0.5.

C=u[i] means that points are colored depends on strongly they belong to each cluster.

In the second for loop, we scattered the clusters with different signs and different colors for each of them.

Lastly, centers are scattered with red color and ‘x’ sign.

Final plot graph is at the next page:



3.4.1 Fuzzy-C With Wine Dataset

In this example, just like the previous GMM example, I have decided the use Wine Dataset from scikit learn again.

Additionaly, PCA and StandartScaler functions are used.

It is mosly same with previous example other than few fixes.

Here is the full code.

fuzzy_wine.py

```python
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
import numpy as np
import skfuzzy as fuzz
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

wine = load_wine()

x = wine.data
y = wine.target

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)
```

```
# Fuzzy-C Parameters
n_clusters = 3
m = 1.7 #
error = 1e-5
maxiter = 2000

data = x_pca.T # convert samples x features to features x samples
# (178, 2) --> (2, 178)

cntr, u, _, _, _, _, fpc = fuzz.cluster.cmeans(data, c=n_clusters, maxiter=maxiter, m=m,
                                               error=error, init=None)

hard_clusters = np.argmax(u,axis=0)

fig, ax = plt.subplots(figsize=(8,6))

for i in range(n_clusters):
    ax.scatter(data[0], data[1], c=u[i], cmap='coolwarm', alpha=0.5, label=f'Fuzzy Cluster: {i+1}')

markers = ['o','s','^']
colors = ['b','g','orange'] # blue, green, orange for clusters 1,2,3

for i in range(n_clusters):
    cluster_points = data[:, hard_clusters == i]
    ax.scatter(cluster_points[0], cluster_points[1], c=colors[i], marker=markers[i],
               edgecolors='k', s=80, label=f'Hard Cluster: {i+1}')

ax.scatter(cntr[:,0], cntr[:,1], c='red', marker='x', s=200, label='Cluster Centers')

ax.set_title("Fuzzy C-Means with Wine Dataset")
ax.set_xlabel("Feature 1")
ax.set_ylabel("Feature 2")
plt.show()
```

By using load_wine function, we loaded the dataset.

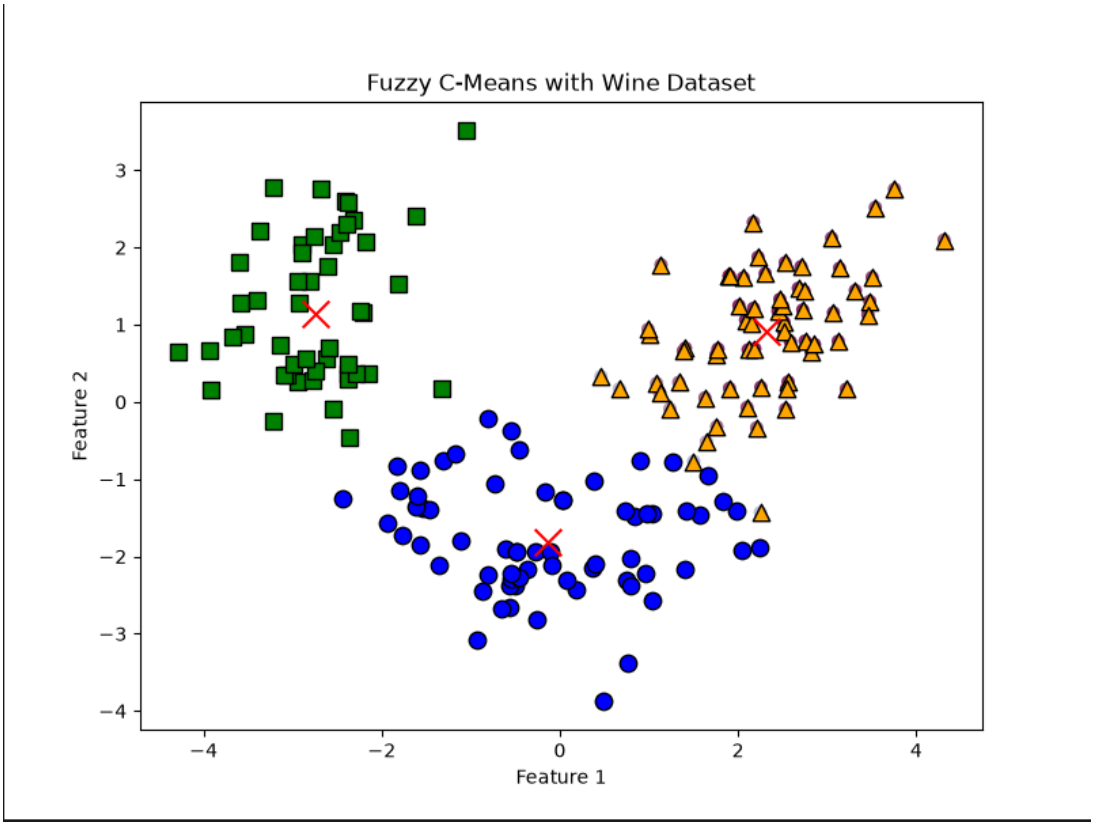
Variable x contains feature values and y contains 3 class samples.

After that, as explained in GMM example before, scaling and PCA operations are performed.

```
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)
```

After this part, remaining code is same with previous example so I wont explain it again.

Final plot graph is at the next page.



4. Conclusion of Clustering

In this report, different clustering algorithms were implemented and analyzed using both synthetic datasets and real-world datasets. The main aim was to understand how clustering algorithms group similar data points without using predefined labels. Since clustering is an unsupervised learning method, the algorithms tried to discover hidden patterns and natural group structures inside the datasets.

First, we have explained hard clustering algorithms which is DBSCAN and K Means Clustering Methods.

Then, Soft Clustering Methods such as Gassian Mixture Model (GMM) and Fuzzy-C Clustering Methods are implemented.

In conclusion, clustering is a powerful unsupervised learning technique for discovering patterns in data. The projects in this report showed how clustering can be applied to artificial datasets, customer segmentation, geographic Airbnb listings, and wine classification data. These examples helped demonstrate both the theoretical ideas and practical applications of clustering algorithms in machine learning.

5. Outlier Detection

Outlier detection is the process of finding data points that are very different from the rest of the dataset. These unusual values are called outliers.

I have implemented 3 examples for Outlier Detection that explains most important outlier detection algorithms.

5.1 Standard Deviation Method

This method is based on the assumption that the data follows a normal distribution.

Data points outside of three standart deviations from the mean are considered as outliers.

First, we find the mean and the standard deviation of a given list.

Then, we define upper and lower bounds using mean and standard deviation.

Lastly, points that are not between these bounds are considered as outliers.

Here is an example given with code:

outlier_detect.py

```python
import pandas as pd
import numpy as np

data = {"salary": [3000, 3200, 3100, 3050, 3300, 10000, 3150, 2900]}

df = pd.DataFrame(data)

mean = df["salary"].mean() # mean of all values given
std_dev = df["salary"].std() # standard deviation of all values given

lower_limit = mean - 2 * std_dev
upper_limit = mean + 2 * std_dev

# if the given values are not between these limits, they are outlier
outlier = df[(df["salary"] < lower_limit) | (df["salary"] > upper_limit)]

print("Mean:", mean)
print("Standard Deviation:", std_dev)
print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)
print("\n")

print("Outliers: ")
print(outlier) # only 10000 is outlier in the dataset
```

Only pandas library is added.

Salary list is defined with numbers and you can clearly see that 10000 is an outlier here but we will implement an algorithm to show it.

First, we used pandas library to create DataFrame.

Mean and standard deviation is found by using mean() and std() functions.

Lower_limit is mean – 2*standard_deviation

Upper_limit is mean + 2*standard_deviation

```
# if the given values are not between these limits, they are outlier
outlier = df[(df["salary"] < lower_limit) | (df["salary"] > upper_limit)]
```

This part allows us to store the values that are smaller than lower limit and values that are bigger than the upper_limit which we call as 'outliers'.

Lastly we printed out all the details, here is the output.

```
Mean: 3962.5
Standard Deviation: 2442.590837614847
Lower Limit: -922.6816752296936
Upper Limit: 8847.681675229695


Outliers:
   salary
5   10000
```

5.2 Interquartile Range (IQR) Method

IQR method detects outliers by using quartiles.

Before working with quartiles, we must sort the data first.

After sorting the data, we divide to 3 parts.

If dataset hast 2n or 2n+1 points:

Q1: Median of the n smallest data points.

Q2: Median of the dataset.

Q3: Median of the n highest data points.

IQR is calculated as IQR = Q3 – Q1

Data points that below Q1 – 1.5 * IQR

Data points that above Q3 + 1.5 * IQR

Are outliers.

Here is an example of IQR Method as code:

iqr_example.py

```python
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

data = [6, 2, 3, 4, 5, 1, 50]
sort_data = np.sort(data)
print(sort_data)
print("\n")
# Here, we sort data and print it make sure it's ready to use IQR Method

Q1 = np.percentile(data, q: 25, method="midpoint")
Q2 = np.percentile(data, q: 50, method="midpoint")
Q3 = np.percentile(data, q: 75, method="midpoint")

print("Q1 25 Percentile of the give data is: ",Q1)
print("Q2 50 Percentile of the give data is: ",Q2)
print("Q3 75 Percentile of the give data is: ",Q3)

IQR = Q3 - Q1 # Interquartile Range
print("Interquartile range is: ",IQR)

# if values are not between limit range, it is an outlier
low_limit = Q1 - 1.5*IQR
high_limit = Q3 + 1.5*IQR
print("Low and high limit is: ",low_limit, ",", high_limit)
print("\n")

outlier = []
for x in data:
    if ((x < low_limit) or (x > high_limit)):
        outlier.append(x)

print("Outliers are: ",outlier)

sns.boxplot(data) # create a graph using seaborn library
plt.show()
```

As addition, seaborn library is added.

First, we defined a data list and sorted it out.

It’s clearly visible that 50 is an outlier in this list.

Q1, Q2 and Q3 is defined by using percentile function from numPy.

IQR is found by using Q3 – Q1.

Low_limit and high_limit variables defined by using formula explained before page.

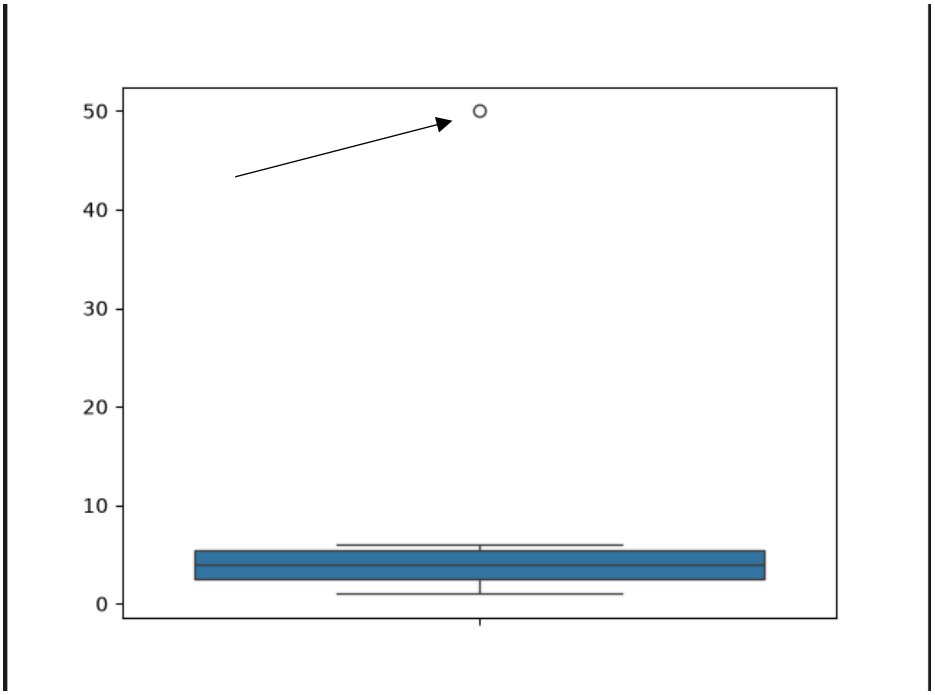
Created an outlier list to store every outlier.

In the for loop, we created an if statement for points thats are outside of the limits.

These values which we call them outliers, then appended to outlier list.

And lastly by using seaborn, we can clearly see which value is an outlier at an graph.

Here is the final output:



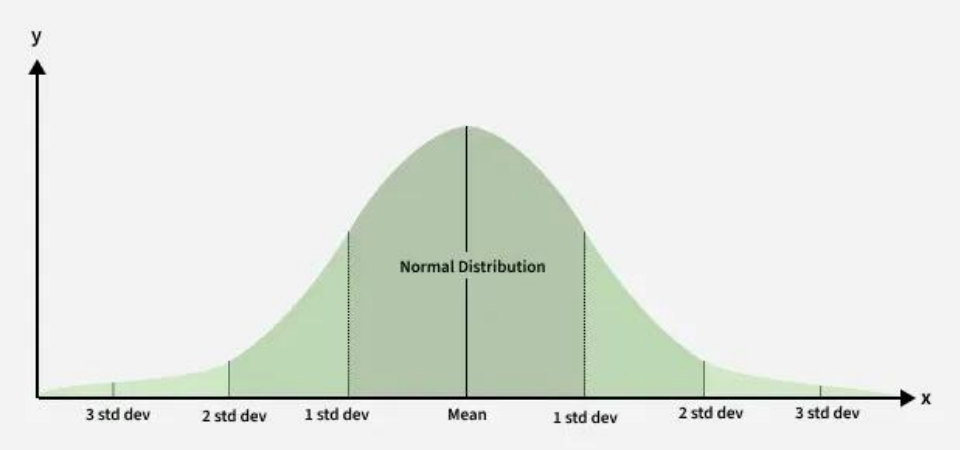
Z-Score Outlier Method

Z-Score Outlier Method is statistical method used to detect values that are very far from the average.

Z-Score is an another outlier detection technique that used Z-Score as a formula.

It can be found by (current point – mean) / standard deviation.

If |Z| > 3, it means that point is considered as an outlier, here is an example graph to explain it.



Now, lets show an example with code:

z_sc0re.py

```python
from matplotlib import pyplot as plt
import pandas as pd
import numpy as np
from scipy.stats import zscore # scipy library helps finding z_score of each value

data = [5, 2, 4.5, 4, 3, 2, 6, 20, 9, 2.5, 3.5, 4.75, 6.5, 2.5, 8, 1]
df = pd.DataFrame(data, columns=['Value'])
print(df.head())

print("-----------------------------")

df['Z-score'] = zscore(df['Value']) # z_score = (current_value - mean) / standard deviation
print(df)
print("\n")


print("Outlier Values:")
outliers = df[df['Z-score'].abs() > 3] # if |z_score| bigger than 3, that value is an outlier
print(outliers)

plt.figure(figsize=(10,6))

plt.scatter(df['Value'].index, df['Z-score'], label='Data Points') # creates data points
plt.scatter(outliers['Value'].index, outliers['Z-score'], color='red', label='Outliers')
# creates outliers but with red color

plt.xlabel('Index Value')
plt.ylabel('Z-score')
plt.legend()
plt.grid(True)
plt.show()

```

New library called scipy added to implement Z Score formula.

Data list is created and by using pandas library, we have transformed data into an DataFrame.

We added a new row called Z-Score the store the Z-Score of each point.

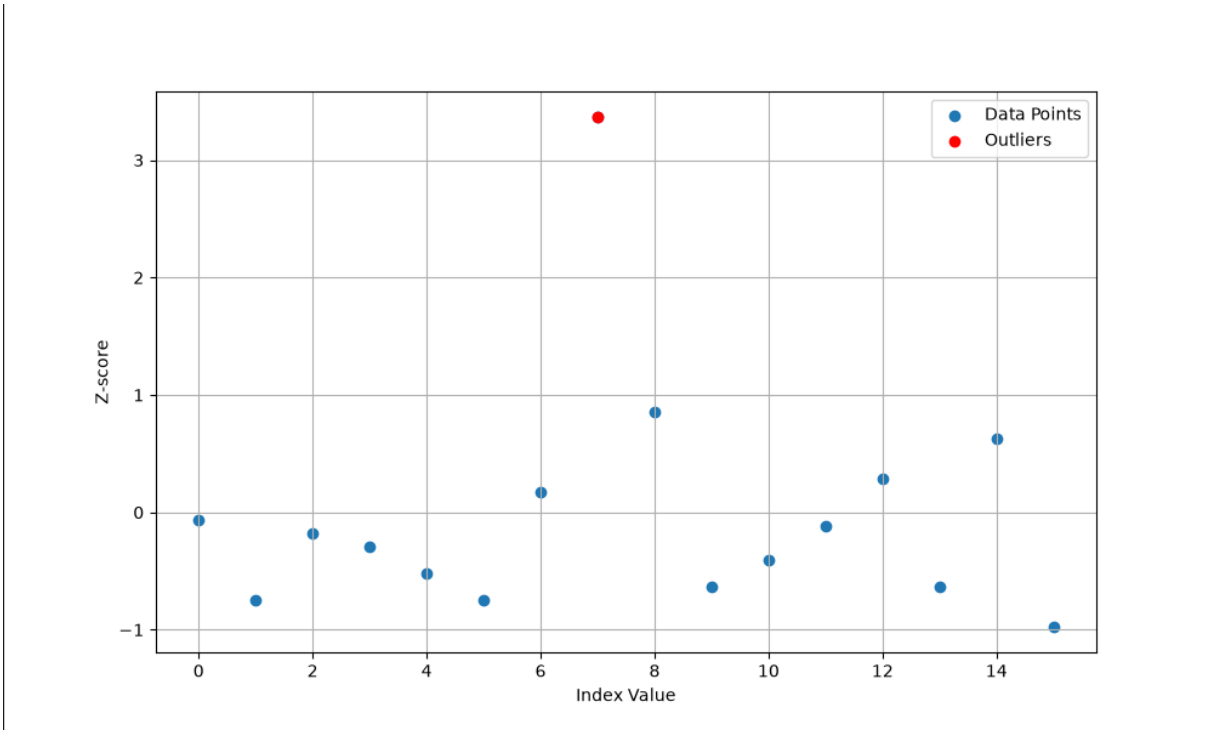
Outliers values are found by using this abs()

```
print("Outlier Values:")
outliers = df[df['Z-score'].abs() > 3] # if |z_score| bigger than 3, that value is an outlier
print(outliers)
```

Abs() means absolute value.

Lastly, we created plot and scatter the data points and outliers.

Here is the graph:



6. Final Conclusion

In this report, different unsupervised learning methods were implemented and analyzed using both synthetic and real-world datasets.

The main focus was on two important machine learning topics: clustering and outlier detection. These methods are useful for discovering hidden patterns, grouping similar data points, and detecting unusual values without using predefined labels.

In Clustering, many examples created with both sample datasets and real life datasets.

The most realistic one was the Prague Airbnb Dataset example that uses DBSCAN as Clustering Method.

4 Methods implemented for clustering.

1 – K Means

2 – DBSCAN

3 – Gaussian Mixture Model

4 – Fuzzy – C

These are one of the most commonly used clustering algorithms.

As outlier detection, 3 methods implemented.

1- Standard Deviation

2 – IQR

3 – Z-Score